



جامعة برج العرب التكنولوجية

Information Technology Department



كلية تكنولوجيا الصناعة والطاقة

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

INTRODUCTION TO C PROGRAMMING ESSENTIALS

First Year – Information Technology Program

spring 2026

LECTURE 6

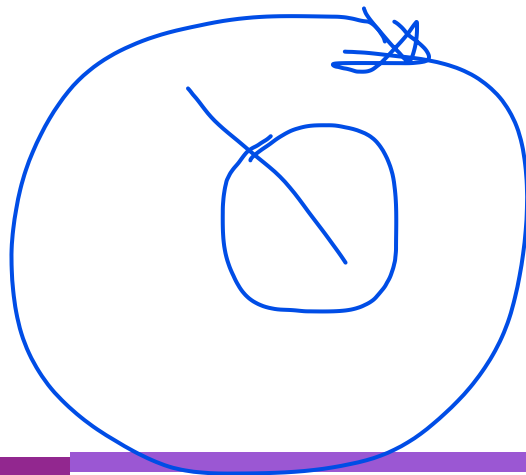
C LANGUAGE BASICS

LOOPS (BREAK AND CONTINUE)

Dr. Dina Abdelhafiz & Dr. Ghada Fathy

BREAK AND CONTINUE IN LOOPS

- Loops can execute a block of code as long as a specified condition is true.
- You have already seen the break statement used with switch statement.
- The break statement can also be used to jump out of a loop.
- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.



BREAK

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    for (i = 0; i < 10; i++) {
```

```
        if (i == 4) {
```

```
            break;
```

```
        }
```

```
        printf("%d\n", i);
```

```
    }
```

```
    return 0;
```

This example stops the loop when **i** is equal to 4 because of the break statement

Output

0

1

2

3

CONTINUE

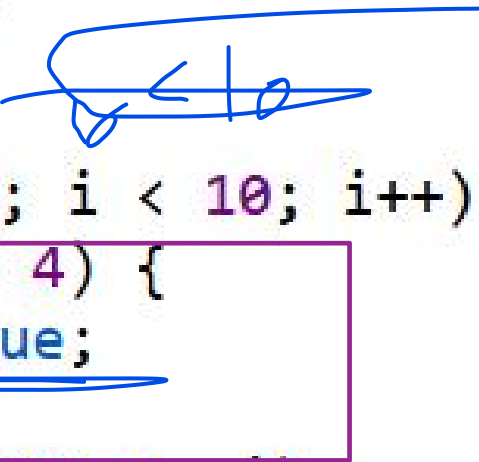
- This example skips the value of 4:

```
#include <stdio.h>

int main() {
    int i;

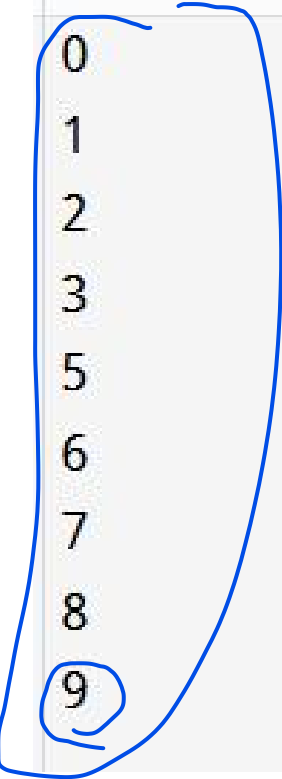
    for (i = 0; i < 10; i++) {
        if (i == 4) {
            continue;
        }
        printf("%d\n", i);
    }

    return 0;
}
```



Output

0
1
2
3
5
6
7
8
9



GOOD TO REMEMBER:

- break = stop the loop completely.
 - continue = skip this round, but keep looping.
- 

COMBINING BREAK AND CONTINUE

```
#include <stdio.h>

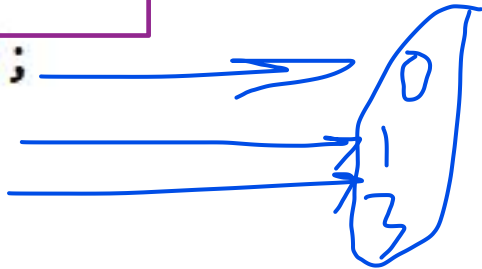
int main() {
    int i;

    for (i = 0; i < 6; i++) {
        if (i == 2) {
            continue;
        }
        if (i == 4) {
            break;
        }
        printf("%d\n", i);
    }

    return 0;
}
```

$i < 6$

0
1
2
3
4



Output

0
1
3

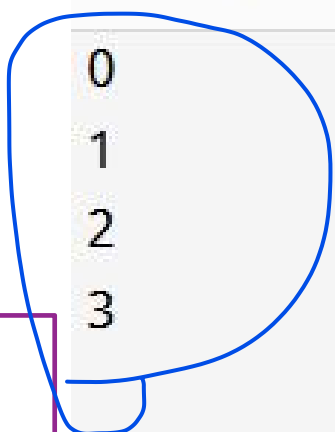
BREAK AND CONTINUE IN WHILE LOOP

```
#include <stdio.h>
```

```
int main() {  
    int i = 0;  
  
    while (i < 10) {  
        if (i == 4) {  
            break; →  
        }  
        printf("%d\n", i);  
        i++;  
    }  
  
    return 0;  
}
```

Output

0
1
2
3

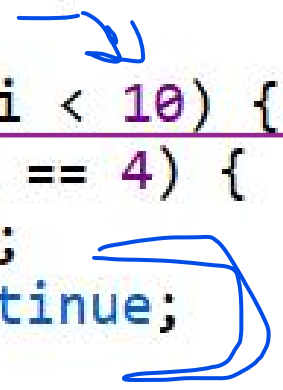


```
#include <stdio.h>
```

```
int main() {  
    int i = 0;  
  
    while (i < 10) {  
        if (i == 4) {  
            i++;  
            continue;  
        }  
        printf("%d\n", i);  
        i++;  
    }  
  
    return 0;  
}
```

Output

0
1
2
3
→ 4
5
6
7
8
9



EXERCISE

What does the break statement do in a loop?

- A. Skips the current iteration and continues with the next
- B. Ends the entire loop
- C. Pauses the loop temporarily →
- D. Restarts the loop from the beginning

What does the continue statement do in a loop?

- A. Ends the entire loop
- B. Skips the current iteration and continues with the next
- C. Pauses the loop temporarily
- D. Ends the current iteration and jumps out of the loop

What will be the output of this code?

```
int i;
for (i = 0; i < 10; i++) {
    if (i == 4) {
        break;
    }
    printf("%d ", i);
}
```

- A. 0 1 2 3 4
- B. 4
- C. 0 1 2 3
- D. 0 1 2 3 5 6 7 8 9

What will be the output of this code?

```
int i;
for (i = 0; i < 10; i++) {
    if (i == 4) {
        continue;
    }
    printf("%d ", i);
}
```

- A. 0 1 2 3 5 6 7 8 9
- B. 0 1 2 3 4
- C. 4 5 6 7 8 9
- D. 1 2 3 4 5



C USER INPUT



- You have already learned that `printf()` is used to **output values** in C.
- To get **user input**, you can use the `scanf()` function:

MULTIPLE INPUTS

```
#include <stdio.h>
```

```
int main() {
```

```
    // Create an int and a char variable
```

```
    int myNum;
```

```
    char myChar;
```

```
    // Ask the user to type a number AND a character
```

```
    printf("Type a number AND a character and press enter: \n");
```

```
    // Get and save the number AND character the user types
```

```
    scanf("%d %c", &myNum, &myChar);
```

```
    // Print the number
```

```
    printf("Your number is: %d\n", myNum);
```

```
    // Print the character
```

```
    printf("Your character is: %c\n", myChar);
```

```
    return 0;
```

```
}
```

Output

Type a number AND a character and press enter:)

5d

Your number is: 5

Your character is: d

```
scanf("%d", &myNum);  
scanf("%c", &myChar);
```

TAKE STRING INPUT

- You can also get a string entered by the user:

```
#include <stdio.h>
```

```
int main() {  
    // Create a string  
    char firstName[30];  
    // Ask the user to input some text (name)  
    printf("Enter your first name and press enter: \n");  
  
    // Get and save the text  
    scanf("%s", firstName);  
  
    // Output the text  
    printf("Hello %s", firstName);  
  
    return 0;  
}
```

Output

```
Enter your first name and press enter:  
Dina Abdelhafiz  
Hello Dina
```

- the scanf() function has some limitations: it considers space (whitespace, tabs, etc) as a terminating character, which means that it can only display a single word (even if you type many words).

Output

Enter your first name and press enter:

Dina Abdelhafiz

hello Dina

- when working with strings, we often use the fgets() function to read a line of text. Note that you must include the following arguments: the name of the string variable, sizeof(string_name), and stdin:

```
#include <stdio.h>

int main() {
    // Create a string
    char fullName[30];

    // Ask the user to input some text (full name)
    printf("Type your full name and press enter: \n");

    // Get the text
    fgets(fullName, sizeof(fullName), stdin);

    // Output the text
    printf("Hello %s", fullName);

    return 0;
}
```

Output

```
Type your full name and press enter:
Ghada Fathy
Hello Ghada Fathy
```

EXERCISE

- Use the `scanf()` function to get a single word as input, and use `fgets()` for multiple words.

Which function is used to get user input in

- A. `printf()`
- B. `scanf()`
- C. `pgets()`
- D. `sgets()`

What is the correct syntax to store an integer input from the user in a variable called `myNum`? 

- A. `scanf("%d", myNum);`
- B. `scanf("%d", &myNum);`
- C. ~~`scanf(myNum, "%d");`~~
- D. ~~`scanf(&myNum, "%d");`~~

Which format specifier is used with `scanf()` to read a single character?

- A. `%d`
- B. `%c`
- C. `%f`
- D. `%s`

Which function should you use if you need to read a full line of text including spaces?

- A. `scanf()`
- B. `gets()`
- C. `fgets()`
- D. `puts()`

EXERCISE

What will the following code output if the user enters John Doe?

```
char name[20];  
scanf("%s", name);  
printf("Hello %s", name);
```

- A. Hello John
- B. Hello John Doe
- C. Hello
- D. Hello Doe



Happy Coding!